

LLM Project Lifecycle Manual

How to prepare the environment and operate LCD-bitmap IDE with Claude Code, Codex, OpenCode, LM Studio and Ollama.

- | | | |
|-----------------------------------|----------------------------|----------------------------|
| 1. What The LLM Should Do | 2. Prepare The Environment | 3. Choose A Connector Path |
| 4. Recommended System Prompt | 5. Read Project State | 6. Edit The Project Safely |
| 7. Validate Measurement Workflows | 8. Compile And Review | 9. Release Handoff |
| 10. Security Rules | | |

1. What The LLM Should Do

Use the LLM as an assistant for structured edits, reviews and repetitive project work. The UI remains the source of truth.

The LLM should read project state first, propose a small change, apply it through MCP or REST, then validate and report the result.

- ☐ Read before write.
- ☐ Change one concept at a time.
- ☐ Validate after each batch.
- ☐ Keep the desktop app visible.

2. Prepare The Environment

1. Install Node.js 22+, Git and a supported LLM client.
2. Run npm ci in the repository.
3. Start the Electron app with npm run electron:dev.
4. Open a project or the bundled demo before calling tools.

```
npm ci
npm run electron:dev
curl http://127.0.0.1:8766/api/health
```

3. Choose A Connector Path

Client	Best path	Fallback
Claude Code	MCP HTTP server	REST through shell commands
Codex	MCP HTTP server or local shell REST calls	Small JSON scripts
OpenCode	MCP HTTP server	curl commands
LM Studio	Tool wrapper calling REST	Manual curl recipes
Ollama	Agent wrapper calling REST	Scripted batch edits

4. Recommended System Prompt

Give the agent explicit guardrails. It should not delete states, overwrite a project or perform broad edits without confirmation.

```
You are operating LCD-bitmap IDE through local MCP/REST.  
First read the project summary and FSM states.  
Before each mutation, describe the intended change.  
Apply small changes only.  
After each mutation, validate and summarize the result.  
Do not delete states unless the user explicitly asks.
```

5. Read Project State

Start every session by reading the current project. This avoids stale assumptions and prevents the model from editing the wrong state, screen or tag.

```
curl http://127.0.0.1:8766/api/project/meta  
curl http://127.0.0.1:8766/api/fsm/states  
curl http://127.0.0.1:8766/api/tags
```

6. Edit The Project Safely

1. Ask for one outcome: add a state, rename a screen, add a tag or connect a transition.
2. Let the agent prepare the payload.
3. Apply the payload through MCP or REST.
4. Read the state again and compare the result.
5. Save the project in the UI after successful batches.

7. Validate Measurement Workflows

For measurement-oriented projects, the LLM should verify that setup, run, result, cancel and error paths all exist. It should also check that key measurement tags have units and precision.

- ❑ Initial state exists.
- ❑ START has a valid path.
- ❑ Result state is reachable.
- ❑ Cancel and error paths are reachable.
- ❑ Measurement tags have names, units and precision.

8. Compile And Review

1. Compile screens after LCD changes.
2. Run Runtime Preview after FSM changes.
3. Review screenshots or exported assets before handing changes to firmware.
4. Commit documentation and project changes together when they describe the same behavior.

9. Release Handoff

A clean handoff contains the .lcdproj file, exported firmware assets, screenshots, validation notes and the exact application version or commit hash used to generate them.

10. Security Rules

- ❑ Do not expose 127.0.0.1 automation ports through a public tunnel.
- ❑ Do not paste private implementation files, device identifiers or keys into prompts.
- ❑ Review every DELETE operation manually.
- ❑ Keep proprietary projects outside the public branch.

Generated from scripts/generate-user-docs.mjs. For automation details see docs/API_MCP_CONNECTORS.md.